



Semester 01 2025/2026

Subject : Database Programming (SECP3623)

Section : Section 1

Task : Project I

Due : 25th November 2025 (10%)

Name	Matric No.
Choh Jing Yi	A23CS0296
Chew Chiu Xian	A23CS0061
Neo Li Xin	A23CS0253

Video presentation link: <https://youtu.be/8Vve4NU31JI>

Index

1.0 Introduction	3
1.1 Background of Domain	3
1.2 System Objectives	3
1.3 Team Members and Roles	3
2.0 SQL Implementation Summary	4
2.1 SQL File Structure	4
2.2 Database Schema (DDL)	4
2.3 Constraints	6
2.4 Data Manipulation (DML)	6
2.5 Data Query (DQL)	7
2.6 Index	30
4.0 Team Work	33
5.0 Conclusion	34

1.0 Introduction

1.1 Background of Domain

This project involves the design and implementation of a backend database for a Simple Project Management System. In any company, tracking projects, assigning tasks, and managing employees is a fundamental need. This system provides a centralized database to handle these operations. The database is designed to store employee records, detailed project information, and the individual tasks that link employees to those projects. This allows for clear reporting on project status, employee workload, and task priorities.

1.2 System Objectives

The primary objectives of this database system are:

- To design a normalized relational database schema to store Employees, Projects, and Tasks.
- To ensure data integrity and consistency through the use of constraints (e.g., PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK).
- To implement SQL statements to perform all necessary CRUD (Create, Read, Update, Delete) operations.
- To demonstrate a wide range of data retrieval (DQL) capabilities, from simple filtering to complex joins and subqueries, to support management reporting.
- To analyse and optimize query performance by implementing BTREE and FULLTEXT indexes.

1.3 Team Members and Roles

Member	Role
Choh Jing Yi	Team Leader & Database Designer (DDL): Responsible for the overall ERD design, schema creation (01_Schema.sql), and final report compilation.
Chew Chiu Xian	Data Manager (DML) : Responsible for populating all tables with data (02_Data.sql) including filtering (WHERE, LIKE), sorting (ORDER BY), and generating statistical reports using aggregation and grouping (GROUP BY, HAVING).
Neo Li Xin	Query Specialist (DQL): Responsible for developing all SELECT queries, including joins, subqueries, aggregations, and functions (03_Queries.sql).

	Responsible for Index by implementing BTREE and TEXT indexes and performing EXPLAIN analysis.
--	--

2.0 SQL Implementation Summary

2.1 SQL File Structure

Our project is organized into three distinct SQL files for clarity and modularity:

1. **01_Schema.sql:** This file contains all DDL (Data Definition Language) statements. It creates the ProjectMgmtDB database, drops any existing tables, and then creates the Employees, Projects, and Tasks tables with all their constraints.
2. **02_Data.sql:** This file contains all DML (Data Manipulation Language) statements. It populates the tables with over 10 sample records each and includes examples of UPDATE and DELETE operations.
3. **03_Queries.sql:** This file contains all DQL (Data Query Language) statements for data retrieval, as well as the EXPLAIN and CREATE INDEX commands for the optimization section.

2.2 Database Schema (DDL)

The following SQL snippets demonstrate the creation of our primary tables and their constraints.

- a) **Employees Table:** This table stores employee records and includes a UNIQUE constraint on email. It serves as the "parent" table for project managers and task assignees.

```
-- Stores all employee records
CREATE TABLE Employees (
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    job_title VARCHAR(50),
    hire_date DATE NOT NULL DEFAULT (CURRENT_DATE)
);
```

Figure 2.2.1 DDL operation of creating Employees Table using CREATE TABLE

- b) **Tasks Table:** This table is the core of the system, linking Projects and Employees. It uses two FOREIGN KEY constraints to ensure referential integrity and CHECK constraints to validate data for status and priority.

```
-- Tasks table
-- Stores individual tasks, linking projects and employees
CREATE TABLE Tasks (
    task_id INT AUTO_INCREMENT PRIMARY KEY,
    project_id INT NOT NULL,
    assignee_id INT, -- Can be NULL if unassigned
    task_name VARCHAR(200) NOT NULL,
    status VARCHAR(20) DEFAULT 'To Do',
    priority VARCHAR(10) DEFAULT 'Medium',

    -- FOREIGN KEY constraints
    FOREIGN KEY (project_id) REFERENCES Projects(project_id),
    FOREIGN KEY (assignee_id) REFERENCES Employees(emp_id),

    -- CHECK constraints
    CHECK (status IN ('To Do', 'In Progress', 'Done', 'Testing')),
    CHECK (priority IN ('High', 'Medium', 'Low'))
);
```

Figure 2.2.2 DDL operation of creating Tasks Table using CREATE TABLE

- c) **Projects Table:** This table stores all project records and includes a UNIQUE constraint on the project name. It utilizes a FOREIGN KEY constraint to assign a manager from the Employees table, ensuring referential integrity.

```
-- Projects table
-- Stores all project records
CREATE TABLE Projects (
    project_id INT AUTO_INCREMENT PRIMARY KEY,
    project_name VARCHAR(150) NOT NULL UNIQUE,
    project_description TEXT, -- For FULLTEXT index demo
    start_date DATE,
    due_date DATE,
    manager_id INT, -- This links to an employee

    -- Add FOREIGN KEY constraint
    FOREIGN KEY (manager_id) REFERENCES Employees(emp_id)
);
```

Figure 2.2.3 DDL operation of creating Projects Table using CREATE TABLE

2.3 Constraints

We used the following constraints to ensure data integrity:

- **PRIMARY KEY:** e.g., emp_id in Employees. Uniquely identifies each row.
- **FOREIGN KEY:** e.g., project_id in Tasks. Ensures that a value in one table matches an existing value in another, preventing orphan records.
- **NOT NULL:** e.g., first_name in Employees. Ensures a column cannot have a NULL value.
- **UNIQUE:** e.g., email in Employees. Ensures all values in a column are different.
- **CHECK:** e.g., status IN ('To Do', 'In Progress', 'Done', 'Testing'). Enforces specific valid values for a column.
- **DEFAULT:** e.g., status in Tasks defaults to 'To Do' if not specified.

2.4 Data Manipulation (DML)

Below are representative examples of the INSERT, UPDATE, and DELETE statements used.

INSERT Example: This statement adds a new employee to the Employees table.

```
-- Insert at least 10 records into Employees
INSERT INTO Employees (first_name, last_name, email, job_title, hire_date) VALUES
('Chew', 'Chiu Xian', 'ali.hassan@comp.com', 'Project Manager', '2022-01-10'),
('Neo', 'Li Xin', 'beth.tan@comp.com', 'Senior Developer', '2022-03-15'),
('Choh', 'Jing Yi', 'chris.lee@comp.com', 'Junior Developer', '2023-05-20'),
('Lau', 'Ya Kai', 'diana.ravi@comp.com', 'QA Analyst', '2022-07-01'),
('Yap', 'Deh Kai', 'evan.wong@comp.com', 'UI/UX Designer', '2022-09-11'),
('Ramah', 'bin Jaafar', 'fara.ahmed@comp.com', 'Project Manager', '2022-02-14'),
('Chong', 'Joe Lim', 'gani.i@comp.com', 'Database Admin', '2023-01-30'),
('Kristen Chang', 'Jing Huan', 'hana.chen@comp.com', 'Senior Developer', '2022-03-16'),
('Javier', 'Kastilanos', 'ian.gomez@comp.com', 'Junior Developer', '2024-01-10'),
('Jia', 'Li', 'jia.li@comp.com', 'QA Analyst', '2023-08-05'),
('Brendan', 'Chia', 'kumar.nair@comp.com', 'Intern', '2024-06-01'),
('Howard', 'Switsland', 'lena.schmidt@comp.com', 'Business Analyst', '2022-04-10');
```

Figure 2.4.1 DML operation using INSERT condition

UPDATE Example: This statement updates a task's status from 'To Do' to 'In Progress' based on its name.

```
-- UPDATE with WHERE
-- Update a task's status from 'To Do' to 'In Progress'
UPDATE Tasks
SET status = 'In Progress'
WHERE task_name = 'Write test cases for login';
```

Figure 2.4.2 DML operation using UPDATE condition

DELETE Example: This statement removes a cancelled project. We must first delete the tasks associated with it from the Tasks table to avoid violating the foreign key constraint.

```
-- DELETE with WHERE
-- Delete the cancelled project.
-- Must delete tasks from child table first due to foreign key constraints.
DELETE FROM Tasks
WHERE project_id = (SELECT project_id FROM Projects WHERE project_name = 'Cancelled Project Alpha');

DELETE FROM Projects
WHERE project_name = 'Cancelled Project Alpha';
```

Figure 2.4.3 DML operation using DELETE condition

2.5 Data Query (DQL)

Below are demonstrations of SQL queries that perform filtering, sorting, aggregation, grouping and filtering groups, numeric and string functions, conditional logic, subqueries, set operations and finally joins operations.

1. Filtering Example (WHERE, AND, OR, NOT, BETWEEN, LIKE, NULL)

a.

```
-- Find 'High' priority tasks that are 'In Progress' OR 'To Do' (AND, OR)
SELECT task_name, status, priority
FROM Tasks
WHERE priority = 'High'
      AND (status = 'In Progress' OR status = 'To Do');
```

Figure 2.5.1 DQL operation using WHERE and AND condition

	task_name	status	priority
►	Develop funds transfer module	In Progress	High
	Test funds transfer security	In Progress	High
	Analyze legacy database	In Progress	High
	Develop ETL scripts	In Progress	High
	API for stock update	To Do	High
	Optimize product search query	To Do	High
	Setup cloud database instance	In Progress	High
	Code Review - Security Audit	To Do	High
	Code Review - Security Audit	To Do	High

Table 2.5.1 Result table showcasing 'High' priority tasks that are 'In Progress' OR 'To Do'

Explanation:

From the Tasks table, the query displays the task_name, status and priority columns, where the priority is 'High' and status is 'In Progress' or 'To Do'.

b.

```
-- Find all employees who are NOT 'Junior Developer' (NOT)
SELECT first_name, last_name, job_title
FROM Employees
WHERE NOT job_title = 'Junior Developer';
```

Figure 2.5.2 DQL operation using WHERE and NOT condition

	first_name	last_name	job_title
►	Chew	Chiu Xian	Project Manager
	Neo	Li Xin	Senior Developer
	Lau	Ya Kai	QA Analyst
	Yap	Deh Kai	UI/UX Designer
	Ramah	bin Jaafar	Project Manager
	Chong	Joe Lim	Database Admin
	Kristen Chang	Jing Huan	Senior Developer
	Jia	Li	QA Analyst
	Brendan	Chia	Intern
	Howard	Switsland	Business Analyst

Table 2.5.2 Result table 2.5.2 showcasing employees who are NOT ‘Junior Developer’

Explanation:

From the Employees table, the query displays the first_name, last_name and job_title columns, where all the job_title are displayed except for ‘Junior Developer’.

c.

```
-- Find all projects starting BETWEEN 2024-01-01 and 2024-06-30 (BETWEEN)
SELECT project_name, start_date
FROM Projects
WHERE start_date BETWEEN '2024-01-01' AND '2024-06-30';
```

Figure 2.5.3 DQL operation using WHERE and BETWEEN condition

	project_name	start_date
►	E-commerce Platform	2024-01-01
	Mobile Banking App	2024-03-01
	Internal Wiki	2024-02-15
	Data Warehouse Migration	2024-05-01
	Inventory Management Tool	2024-06-15

Table 2.5.3 Result table showcasing all projects between '2024-01-01' and '2024-06-30'

Explanation:

From the Projects table, the query displays the project_name and start_date columns, where the start_date is between '01-01-2024' and '30-06-2024'.

d.

```
-- Find all tasks with 'database' in the name (LIKE)
SELECT task_name, project_id
FROM Tasks
WHERE task_name LIKE '%database%';
```

Figure 2.5.4 DQL operation using WHERE and LIKE condition

	task_name	project_id
▶	Design database schema	1
	Analyze legacy database	4
	Setup cloud database instance	4

Table 2.5.4 Result table showcasing all tasks with 'database' in the name

Explanation:

From the Tasks table, the query displays the task_name and project_id columns, where the task_name contains the word 'database'.

e.

```
-- Find all unassigned tasks (IS NULL)
SELECT task_name, status, priority
FROM Tasks
WHERE assignee_id IS NULL;
```

Figure 2.5.5 DQL operation using WHERE and IS NULL condition

	task_name	status	priority
►	Unassigned bug fix	To Do	Low

Table 2.5.5 Result table showcasing tasks that are unassigned

Explanation:

From the Tasks table, the query displays the task_name, status and priority columns, where the task has not been assigned to an employee.

2. Sorting (ORDER BY, LIMIT)

a.

```
-- Get all employees, sorted by hire date (newest first) (ORDER BY)
SELECT first_name, last_name, hire_date
FROM Employees
ORDER BY hire_date DESC;
```

Figure 2.5.6 DQL operation using ORDER BY clause

	first_name	last_name	hire_date
►	Brendan	Chia	2024-06-01
	Javier	Kastilanos	2024-01-10
	Jia	Li	2023-08-05
	Choh	Jing Yi	2023-05-20
	Chong	Joe Lim	2023-01-30
	Yap	Deh Kai	2022-09-11
	Lau	Ya Kai	2022-07-01
	Howard	Switsland	2022-04-10
	Kristen Chang	Jing Huan	2022-03-16
	Neo	Li Xin	2022-03-15
	Ramah	bin Jaafar	2022-02-14
	Chew	Chiu Xian	2022-01-10

Table 2.5.6 Result table showcasing all employees sorted by hiring date in descending order

Explanation:

From the Employees table, the query displays the first_name, last_name and hire_date columns, where the list is descendingly ordered by their hire_date.

b.

```
-- Get the 5 most recently hired employees (ORDER BY, LIMIT)
SELECT first_name, last_name, hire_date
FROM Employees
ORDER BY hire_date DESC
LIMIT 5;
```

Figure 2.5.7 DQL operation using ORDER BY and LIMIT clause

	first_name	last_name	hire_date
►	Brendan	Chia	2024-06-01
	Javier	Kastilanos	2024-01-10
	Jia	Li	2023-08-05
	Choh	Jing Yi	2023-05-20
	Chong	Joe Lim	2023-01-30

Table 2.5.7 Result table showcasing the 5 most recently hired employees

Explanation:

From the Employees table, the query displays the first_name, last_name and hire_date columns, where the list is descendingly-ordered and only 5 rows are displayed with the LIMIT function.

3. Aggregation (COUNT, SUM, AVG, MAX, MIN)

a.

```
-- Get the total number of employees, total project budget (SUM), and avg budget (AVG)
SELECT
    COUNT(*) AS total_employees
FROM Employees;

SELECT
    SUM(budget) AS total_all_budgets,
    AVG(budget) AS average_project_budget
FROM Projects;
```

Figure 2.5.8 DQL operation using COUNT, SUM and AVG function

	total_employees
►	12

Table 2.5.8 Result table showcasing total employees

	total_all_budgets	average_project_budget
▶	1275000.00	141666.666667

Table 2.5.9 Result table showcasing total project budget and average budget

Explanation:

From the Employees table, the SUM function counts the total number of employees. From the Projects table, the total project budget is found using SUM function and then the average of that result is calculated with the AVERAGE function.

b.

```
-- Get the earliest project start date (MIN) and latest due date (MAX)
SELECT
    MIN(start_date) AS first_project_start,
    MAX(due_date) AS last_project_due
FROM Projects;
```

Figure 2.5.9 DQL operation using MIN and MAX function

	first_project_start	last_project_due
▶	2024-01-01	2025-10-31

Table 2.5.10 Result table showcasing the earliest project start date and latest due date

Explanation:

From the Projects table, the query gets the minimum date and maximum date and displays them in the first_project_start and last_project_date columns respectively.

4. Grouping and filtering groups (GROUP BY, HAVING)

a.

```
-- Count of tasks per project (GROUP BY)
SELECT project_id, COUNT(task_id) AS number_of_tasks
FROM Tasks
GROUP BY project_id;
```

Figure 2.5.10 DQL operation using GROUP BY clause

	project_id	number_of_tasks
▶	1	8
	2	4
	3	2
	4	3
	5	2
	8	2

Table 2.5.11 Result table showcasing number of tasks completed for each project

Explanation:

From the Tasks table, the query displays project_id and number_of_tasks columns, where number of tasks for each project is found using GROUP BY function.

b.

```
-- Show employees who are assigned MORE THAN 2 tasks (GROUP BY, HAVING)
SELECT assignee_id, COUNT(task_id) AS task_count
FROM Tasks
WHERE assignee_id IS NOT NULL
GROUP BY assignee_id
HAVING task_count > 2;
```

Figure 2.5.11 DQL operation using GROUP BY and HAVING clause

	assignee_id	task_count
▶	2	4
	5	3
	7	3
	8	4

Table 2.5.12 Result table showcasing employees who are assigned to more than 2 tasks

Explanation:

From the Tasks table, the query displays assignee_id and task_count columns, where the employees are assigned to more than two tasks.

5. Numeric and string functions (ROUND, TRUNCATE, UPPER, LENGTH, CONCAT, SUBSTR)

a.

```
-- Show average project budget, rounded to 2 decimal places (ROUND)
SELECT ROUND(AVG(budget), 2) AS rounded_avg_budget
FROM Projects;
```

Figure 2.5.12 DQL operation using ROUND function

	rounded_avg_budget
▶	141666.67

Table 2.5.13 Result table showcasing the average project budget (2 d.p.)

Explanation:

From the Projects table, the query displays the average project budget and rounds it to 2 decimal places using the ROUND function.

b.

```
-- Show average project budget, truncated (TRUNCATE)
SELECT TRUNCATE(AVG(budget), 0) AS truncated_avg_budget
FROM Projects;
```

Figure 2.5.13 DQL operation using TRUNCATE function

	truncated_avg_budget
▶	141666

Table 2.5.14 Result table showcasing the truncated average project budget (0 d.p.)

Explanation:

From the Projects table, the query displays the average project budget and truncates it to zero decimal places with the TRUNCATE function.

c.

```
-- Get employee full names in uppercase (CONCAT, UPPER)
SELECT UPPER(CONCAT(first_name, ' ', last_name)) AS full_name, job_title
FROM Employees;
```

Figure 2.5.14 DQL operation using CONCAT and UPPER function

	full_name	job_title
►	CHEW CHIU XIAN	Project Manager
	NEO LI XIN	Senior Developer
	CHOH JING YI	Junior Developer
	LAU YA KAI	QA Analyst
	YAP DEH KAI	UI/UX Designer
	RAMAH BIN JAAFAR	Project Manager
	CHONG JOE LIM	Database Admin
	KRISTEN CHANG JING HUAN	Senior Developer
	JAVIER KASTILANOS	Junior Developer
	JIA LI	QA Analyst
	BRENDAN CHIA	Intern
	HOWARD SWITSLAND	Business Analyst

Table 2.5.15 Result table showcasing the full name of employees in capital letters

Explanation:

From the Employees table, the query displays the full_name and job_title columns, where the first_name and last_name are concatenated using the CONCAT function and converted to uppercase using the UPPER function.

d.

```
-- Get the length of each project's name (LENGTH)
SELECT project_name, LENGTH(project_name) AS name_length
FROM Projects;
```

Figure 2.5.15 DQL operation using LENGTH function

	project_name	name_length
▶	AI Chatbot	10
	Customer Feedback System	24
	Data Warehouse Migration	24
	E-commerce Platform	19
	HR Payroll System	17
	Internal Wiki	13
	Inventory Management Tool	25
	Mobile Banking App	18
	Website Redesign	16

Table 2.5.16 Result table showcasing the length of each project's name

Explanation:

From the Projects table, the query displays the project_name and name_length columns, where the length of the project name is calculated using the LENGTH function.

e.

```
-- Get the email domain for each employee (SUBSTR, INSTR)
SELECT email, SUBSTR(email, INSTR(email, '@') + 1) AS email_domain
FROM Employees;
```

Figure 2.5.16 DQL operation using SUBSTR and INSTR function

	email	email_domain
▶	ali.hassan@comp.com	comp.com
	beth.tan@comp.com	comp.com
	chris.lee@comp.com	comp.com
	diana.ravi@comp.com	comp.com
	evan.wong@comp.com	comp.com
	fara.ahmed@comp.com	comp.com
	gani.i@comp.com	comp.com
	hana.chen@comp.com	comp.com
	ian.gomez@comp.com	comp.com
	jia.li@comp.com	comp.com
	kumar.nair@comp.com	comp.com
	lena.schmidt@comp.com	comp.com

Table 2.5.17 Result table showcasing the table domain of each employee's email address

Explanation:

From the Employees table, the query displays the email and email_domain columns, where it extracts the domain part of each employee's email address which are after the @ symbol.

6. Conditional logic (CASE WHEN)

a.

```
-- Categorize tasks by priority level ('Urgent' or 'Normal')
SELECT task_name, priority,
CASE
    WHEN priority = 'High' THEN 'Urgent'
    WHEN priority = 'Medium' THEN 'Normal'
    ELSE 'Low Priority'
END AS priority_level
FROM Tasks;
```

Figure 2.5.17 DQL operation using CASE WHEN condition

	task_name	priority	priority_level
►	Design database schema	High	Urgent
	Develop user login page	Medium	Normal
	Write test cases for login	Medium	Normal
	Create wireframes for homepage	High	Urgent
	Setup mobile app repository	High	Urgent
	Develop funds transfer module	High	Urgent
	Test funds transfer security	High	Urgent
	Install CMS software	Medium	Normal
	Create user documentation page	Low	Low Priority
	Analyze legacy database	High	Urgent
	Develop ETL scripts	High	Urgent
	API for stock update	High	Urgent
	Design inventory dashboard	Medium	Normal
	Finalize new logo design	Medium	Normal
	Implement new CSS styles	Medium	Normal
	Optimize product search query	High	Urgent
	Refactor authentication code	Medium	Normal
	Setup cloud database instance	High	Urgent
	Unassigned bug fix	Low	Low Priority
	Code Review - Security Audit	High	Urgent
	Code Review - Security Audit	High	Urgent

Table 2.5.18 Result table showcasing categorised tasks by priority level ('Urgent' or 'Normal')

Explanation:

From the Tasks table, the query displays the task_name, priority and priority_level columns with the following case: if priority = 'High' then priority_level = 'Urgent'; if priority = 'Medium' then priority_level = 'Normal'; else priority_level = 'Low Priority'.

7. Subqueries (single-row, multiple-row, correlated)

a.

```
-- (Single-row) Find tasks that are in the same project as task_id 10
SELECT task_name, project_id
FROM Tasks
WHERE project_id = (
    SELECT project_id
    FROM Tasks
    WHERE task_id = 10
);
```

Figure 2.5.18 DQL operation demonstrating a single-row subquery

	task_name	project_id
►	Analyze legacy database	4
	Develop ETL scripts	4
	Setup cloud database instance	4

Table 2.5.19 Result table showcasing tasks that are in the same project as task_id 10

Explanation:

From the Tasks table, the query displays the task_name and project_id columns, where the task_id is 10.

b.

```
-- (Multiple-row - NOT IN) Find employees who are NOT project managers
SELECT first_name, last_name, job_title
FROM Employees
WHERE emp_id NOT IN (
    SELECT DISTINCT manager_id
    FROM Projects
    WHERE manager_id IS NOT NULL
);
```

Figure 2.5.19 DQL operation demonstrating a multiple-row subquery

	first_name	last_name	job_title
►	Neo	Li Xin	Senior Developer
	Choh	Jing Yi	Junior Developer
	Lau	Ya Kai	QA Analyst
	Yap	Deh Kai	UI/UX Designer
	Chong	Joe Lim	Database Admin
	Kristen Chang	Jing Huan	Senior Developer
	Javier	Kastlanos	Junior Developer
	Jia	Li	QA Analyst
	Brendan	Chia	Intern
	Howard	Switsland	Business Analyst

Table 2.5.20 Result table showcasing employees who are NOT 'Project Manager'

Explanation:

From the Projects table, the query displays the first_name, last_name and job_title columns, where the employees are not project managers. The SELECT DISTINCT function helps to ensure each manager_id only appears once.

c.

```
-- (Correlated) Find projects that have at least one 'High' priority task
SELECT p.project_name
FROM Projects p
WHERE EXISTS (
    SELECT 1
    FROM Tasks t
    WHERE t.project_id = p.project_id AND t.priority = 'High'
);
```

Figure 2.5.20 DQL operation demonstrating a correlated subquery

	project_name
▶	E-commerce Platform
	Mobile Banking App
	Data Warehouse Migration
	Inventory Management Tool

Table 2.5.21 Result table showcasing projects that have at least one 'High' priority task

Explanation:

From the Projects table, the query displays the project_name column, where the Projects project_id is the same as Tasks project_id and has at least one 'High' priority task.

8. Set operations (UNION, NOT EXISTS)

a.

```
-- Get a combined list of all Employee names and Project names (UNION)
SELECT first_name FROM Employees
UNION
SELECT project_name FROM Projects;
```

Figure 2.5.21 DQL operation using UNION operator

	first_name
▶	Chew
	Neo
	Choh
	Lau
	Yap
	Ramah
	Chong
	Kristen Chang
	Javier
	Jia
	Brendan
	Howard
	AI Chatbot
	Customer Feedback System
	Data Warehouse Migration
	E-commerce Platform
	HR Payroll System
	Internal Wiki
	Inventory Management Tool
	Mobile Banking App
	Website Redesign

Table 2.5.22 Result table showcasing a list of employee and project names in a single column

Explanation:

From the Employees and Projects table, the query displays the first_name column, where the lists consist of Employees first_name and Projects project_name using the UNION function.

b.

```
-- Find employees with ZERO assigned tasks
SELECT e.first_name, e.last_name
FROM Employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM Tasks t
    WHERE t.assignee_id = e.emp_id
);
```

Figure 2.5.22 DQL operation using NOT EXISTS operator

	first_name	last_name
►	Chew	Chiu Xian
	Ramah	bin Jaafar
	Brendan	Chia
	Howard	Switsland

Table 2.5.23 Result table showcasing employees with no assigned tasks

Explanation:

From the Employees table, the query will display the first_name and last_name columns, who do not have any assigned tasks.

9. Joins (NATURAL, INNER, LEFT OUTER, SELF)

a.

```
-- (NATURAL JOIN) List all employees and the tasks they have
SELECT *
FROM Employees
NATURAL JOIN Tasks
LIMIT 10
```

Figure 2.5.23 DQL operation using NATURAL JOIN operator

	emp_id	first_name	last_name	email	job_title	hire_date	task_id	project_id	assignee_id	task_name	status	priority
▶	12	Howard	Switsland	lena.schmidt@comp.com	Business Analyst	2022-04-10	1	1	2	Design database schema	Done	High
	11	Brendan	Chia	kumar.nair@comp.com	Intern	2024-06-01	1	1	2	Design database schema	Done	High
	10	Jia	Li	jia.li@comp.com	QA Analyst	2023-08-05	1	1	2	Design database schema	Done	High
	9	Javier	Kastlanos	ian.gomez@comp.com	Junior Developer	2024-01-10	1	1	2	Design database schema	Done	High
	8	Kristen Chang	Jing Huan	hana.chen@comp.com	Senior Developer	2022-03-16	1	1	2	Design database schema	Done	High
	7	Chong	Joe Lim	gani.i@comp.com	Database Admin	2023-01-30	1	1	2	Design database schema	Done	High
	6	Ramah	bin Jaafar	fara.ahmed@comp.com	Project Manager	2022-02-14	1	1	2	Design database schema	Done	High
	5	Yap	Deh Kai	evan.wong@comp.com	UI/UX Designer	2022-09-11	1	1	2	Design database schema	Done	High
	4	Lau	Ya Kai	diana.ravi@comp.com	QA Analyst	2022-07-01	1	1	2	Design database schema	Done	High
	3	Choh	Jing Yi	chris.lee@comp.com	Junior Developer	2023-05-20	1	1	2	Design database schema	Done	High

Table 2.5.24 Result table of 2 tables joined (Employees and Tasks)

Explanation:

From the Employees and Tasks table, the query will display all the columns from the two tables using the NATURAL JOIN function. This operator is usually not recommended because of its unpredictable behaviour and inconsistent table results.

b.

```
-- (INNER JOIN - 3 Tables) Get a full report: Employee, Task, and Project
SELECT
    e.first_name,
    e.last_name,
    t.task_name,
    t.status,
    p.project_name
FROM Employees e
JOIN Tasks t ON e.emp_id = t.assignee_id
JOIN Projects p ON t.project_id = p.project_id
ORDER BY e.first_name;
```

Figure 2.5.24 DQL operation demonstrating INNER JOIN operation

	first_name	last_name	task_name	status	project_name
►	Choh	Jing Yi	Develop user login page	In Progress	E-commerce Platform
	Choh	Jing Yi	Create user documentation page	To Do	Internal Wiki
	Chong	Joe Lim	Install CMS software	Done	Internal Wiki
	Chong	Joe Lim	Analyze legacy database	In Progress	Data Warehouse Migration
	Chong	Joe Lim	Setup cloud database instance	In Progress	Data Warehouse Migration
	Javier	Kastilanos	Develop funds transfer module	In Progress	Mobile Banking App
	Javier	Kastilanos	Implement new CSS styles	To Do	Website Redesign
	Jia	Li	Test funds transfer security	In Progress	Mobile Banking App
	Kristen Chang	Jing Huan	Setup mobile app repository	Done	Mobile Banking App
	Kristen Chang	Jing Huan	Develop ETL scripts	In Progress	Data Warehouse Migration
	Kristen Chang	Jing Huan	Refactor authentication code	To Do	Mobile Banking App
	Kristen Chang	Jing Huan	Code Review - Security Audit	To Do	E-commerce Platform
	Lau	Ya Kai	Write test cases for login	In Progress	E-commerce Platform
	Neo	Li Xin	Design database schema	Done	E-commerce Platform
	Neo	Li Xin	API for stock update	To Do	Inventory Management Tool
	Neo	Li Xin	Optimize product search query	To Do	E-commerce Platform
	Neo	Li Xin	Code Review - Security Audit	To Do	E-commerce Platform
	Yap	Deh Kai	Create wireframes for homepage	Done	E-commerce Platform
	Yap	Deh Kai	Design inventory dashboard	In Progress	Inventory Management Tool
	Yap	Deh Kai	Finalize new logo design	Done	Website Redesign

Table 2.5.25 Result table of 3 tables joined (Employees, Tasks and Projects)

Explanation:

From the Employees, Tasks and Projects table, the query display the first_name, last_name, task_name, status and project_name columns, where the Employee emp_id is equal to Tasks assignee_id, Tasks project_id is equal to Projects project_id and ordered by their first name.

c.

```

-- (LEFT OUTER JOIN) List ALL employees and the number of tasks they have
-- (Employees with 0 tasks will show 0)
SELECT
    e.first_name,
    e.last_name,
    COUNT(t.task_id) AS task_count
FROM Employees e
LEFT JOIN Tasks t ON e.emp_id = t.assignee_id
GROUP BY e.emp_id, e.first_name, e.last_name
ORDER BY task_count DESC;

```

Figure 2.5.25 DQL operation demonstrating LEFT OUTER JOIN operation using LEFT JOIN operator

	first_name	last_name	task_count
►	Neo	Li Xin	4
	Kristen Chang	Jing Huan	4
	Yap	Deh Kai	3
	Chong	Joe Lim	3
	Choh	Jing Yi	2
	Javier	Kastilanos	2
	Lau	Ya Kai	1
	Jia	Li	1
	Chew	Chiu Xian	0
	Ramah	bin Jaafar	0
	Brendan	Chia	0
	Howard	Switsland	0

Table 2.5.26 Result table showcasing all employees and their number of tasks

Explanation:

From the Tasks and Employees table, the query displays the first_name, last_name and task_count columns, where Employees emp_id is joined with Tasks assignee_id using LEFT JOIN operator, and descendingly ordered by task_count.

d.

```
-- (SELF JOIN) Find pairs of employees who have the same job title
SELECT
    a.first_name AS emp1_first, a.last_name AS emp1_last,
    b.first_name AS emp2_first, b.last_name AS emp2_last,
    a.job_title
FROM Employees a
JOIN Employees b ON a.job_title = b.job_title AND a.emp_id < b.emp_id
ORDER BY a.job_title;
```

Figure 2.5.26 DQL operation demonstrating SELF JOIN operation

	emp1_first	emp1_last	emp2_first	emp2_last	job_title
►	Choh	Jing Yi	Javier	Kastilanos	Junior Developer
	Chew	Chiu Xian	Ramah	bin Jaafar	Project Manager
	Lau	Ya Kai	Jia	Li	QA Analyst
	Neo	Li Xin	Kristen Chang	Jing Huan	Senior Developer

Table 2.5.27 Result table showcasing pairs of employees who have the same job title

Explanation:

From the Employees table, the query displays the emp1_first, emp1_last, emp2_first, emp2_last and job_title columns, where the employees with the same job are paired together.

2.6 Index

1. BTREE Index

Before:

```
-- 1. BTREE Index (for numeric/date columns)

-- Query to analyze: Find employees hired in a specific year.
-- Run this BEFORE creating the index to see the "Full Table Scan".
EXPLAIN SELECT *
FROM Employees
WHERE hire_date BETWEEN '2022-01-01' AND '2022-12-31';
```

Figure 2.6.1 SQL Code to run EXPLAIN function before BTREE Index creation

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	Employees	NULL	ALL	NULL	NULL	NULL	NULL	12	11.11	Using where

Table 2.6.1 EXPLAIN output of before using BTREE Index

After:

```
-- Create the BTREE index (MySQL's default)
CREATE INDEX idx_btree_hire_date
ON Employees(hire_date);

-- Run this AFTER creating the index to see the plan change.
-- It should now use the index ("idx_btree_hire_date")
EXPLAIN SELECT *
FROM Employees
WHERE hire_date BETWEEN '2022-01-01' AND '2022-12-31';
```

Figure 2.6.2 SQL Code to run EXPLAIN function after BTREE Index creation

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	Employees	NULL	range	idx_btree_hire_date	idx_btree_hire_date	3	NULL	7	100.00	Using index condition

Table 2.6.2 EXPLAIN output of after using BTREE Index

Comparison:

In the type column, it changes from ALL to range, indicating that MySQL is now performing index range scan. In the possible_keys and key columns, it changes from NULL to idx_btree_hire_date, showcasing that the query is able to utilize the BTREE index after its creation. The key_len also changes to 3, which indicates MySQL is using 3 bytes of storage for the hire_date attribute. Besides, the rows column changes from 12 to 7, showing that the number of rows searched is actually lesser. The filtered column changes from 11.11 to 100.00, which indicates that all rows that satisfy the index condition also satisfy the WHERE clause condition.

2. TEXT Index

Before:

```
-- 2. TEXT Index (for text search)

-- Query to analyze (Before): Using LIKE on a TEXT column is slow.
-- Run this BEFORE the index. It will show a "Full Table Scan".
EXPLAIN SELECT project_id, project_description
FROM Projects
WHERE project_description LIKE '%database%';
```

Figure 2.6.3 SQL Code to run EXPLAIN function before FULLTEXT Index creation

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	Projects	NULL	ALL	NULL	NULL	NULL	NULL	9	11.11	Using where

Table 2.6.3 EXPLAIN output of before using FULLTEXT Index

After:

```
-- Create the FULLTEXT index
CREATE FULLTEXT INDEX idx_text_description
ON Projects(project_description);

-- Query (After): Use the MATCH...AGAINST syntax.
-- Run this AFTER the index. It will show a "Full-text search".
EXPLAIN SELECT project_id, project_description
FROM Projects
WHERE MATCH(project_description) AGAINST('database' IN NATURAL LANGUAGE MODE);
```

Figure 2.6.4 SQL Code to run EXPLAIN function after FULLTEXT Index creation

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	Projects	NULL	fulltext	idx_text_description	idx_text_description	0	const	1	100.00	Using where; Ft_hints: sorted

Table 2.6.4 EXPLAIN output of after using FULLTEXT Index

Comparison:

In the type column, it changes from ALL to fulltext, indicating that MySQL is now performing full-text index search. In the possible_keys and key columns, it changes from NULL to idx_txt_description, showcasing that the query is able to utilize the TEXT index after its creation. The key_len also changes from NULL to 0, which is expected since it uses internal data structure. Besides, the rows column changes from 9 to 1, showing that the number of rows searched is actually lesser. The filtered column changes from 11.11 to 100.00, showcasing that all rows that matching the full-text search also satisfies the query.

4.0 Team Work

Each group member completed their assigned tasks through effective collaboration and communication. Every member contributed to both the implementation and the documentation of the project. The roles and learning outcomes are as follows:

CHOH JING YI - GROUP LEADER

As the group leader, I held team discussions, organized task distribution, and ensured all project requirements were met before submission. I was responsible for Part 1 (DDL) and the final report compilation. My tasks included defining the project background and objectives, designing the database schema, and implementing all CREATE TABLE statements with necessary constraints (PK, FK, UNIQUE). I also handled structural changes using ALTER and DROP, and wrote the final Conclusion for the report.

I learned how to design a normalized database structure that supports data integrity and how to effectively lead a team to merge different SQL components into a unified system.

CHEW CHIU XIAN - DATA MANAGER

My primary role focused on Part 2 (DML), managing the data flow and basic reporting. I was responsible for populating the database with realistic sample data using INSERT statements. I implemented key data manipulation operations (UPDATE, DELETE) to manage records. Additionally, I developed the fundamental Data Query (DQL) logic, including filtering (WHERE, LIKE), sorting (ORDER BY), and generating statistical reports using aggregation and grouping (GROUP BY, HAVING).

I gained a strong understanding of how to manipulate data safely and how to use aggregate functions to extract meaningful summary statistics from raw data.

NEO LI XIN - QUERY SPECIALIST

I developed the advanced data retrieval features including conditional logic (CASE WHEN), subqueries (correlated and multiple-row), and set operations. I implemented JOINS (Inner, Left, Self) and created demonstration tables to properly showcase them. Finally, I handled the system optimization by implementing BTREE indexes for dates and FULLTEXT indexes for text searching, then using the EXPLAIN function to analyze the performance improvements.

I learned the use of complex joins to link multiple entities and learned how to interpret query execution plans to optimize database search speeds using specific indexing strategies.

5.0 Conclusion

We collaborated and successfully developed a project management system but we encountered some technical challenges such as referential integrity and foreign key constraints. When we were managing FOREIGN KEY constraints during data deletion on the Project table, it resulted in an error because related records still existed in the Tasks table. We learned that we must strictly follow the order of operations, deleting child records (Tasks) before parent records (Projects) to maintain database integrity. Other than that, we faced challenges when constructing the 3-table join to link Employees, Tasks, and

Projects using INNER JOIN. Ensuring the correct ON clauses were used to match emp_id, assignee_id, and project_id required careful logic to avoid missing data.

We propose some suggestions for improvement to further enhance the functionality and efficiency of this system. First, we could introduce Stored Procedures to automate repetitive complex tasks, such as a procedure to 'Onboard New Project' that automatically inserts a project and assigns default initial tasks to a manager. In addition, implementing 'Triggers' would allow for automatic data handling. For example, a trigger could automatically update a Project's status to 'Completed' the moment all associated Tasks are marked as 'Done,' reducing manual administrative work.